

รายงานโปรเจกต์รายวิชา Programming Methodology

ชื่อโปรเจกต์: Kawaii Gacha Pet Sanctuary

จัดทำโดย: ชญญปัญ จวงจันทร์เพ็ญ 6831317021 , พรหมภัสสร ปิริยะวินิต 6831349121

1. ภาพรวมของโปรแกรม (Project Overview)

Kawaii Gacha Pet Sanctuary คือแอปพลิเคชันเกมจำลองการเลี้ยงสัตว์เสมือนจริง (Virtual Pet) ที่ผสมผสานระบบการสุ่มตัวละคร (Gacha System) โดยมีรายละเอียดภาพรวมดังนี้:

- เป้าหมายของเกม: ผู้เล่นจะสวมบทบาทเป็นผู้ดูแลสัตว์เลี้ยง ต้องคอยเอาใจใส่ให้สัตว์เลี้ยงมีชีวิตรอดและมีความสุข
- ระบบเศรษฐกิจ: ผู้เล่นต้องบริหารจัดการเงินเหรียญ (Coins) ภายในเกม ซึ่งหาได้จากการเล่นกับสัตว์เลี้ยง
- การใช้จ่าย: ทรัพยากรเงินสามารถนำไปใช้ซื้ออาหารประจำวัน หรือนำไปใช้สุ่มกาชาเพื่อค้นหาสัตว์เลี้ยงระดับตำนาน (Mythic Pet)

2. คู่มือและวิธีการใช้งานโปรแกรม (User Manual)

การออกแบบหน้าจอผู้ใช้งาน (User Interface) แบ่งออกเป็น 4 ส่วนหลัก เพื่อให้ง่ายต่อการโต้ตอบ ดังนี้:

- ส่วนที่ 1: แถบเมนูด้านบน (Menu Bar)
 - File -> Save: ทำหน้าที่บันทึกข้อมูลความคืบหน้าทั้งหมด (สถานะสัตว์เลี้ยง, เลเวล, จำนวนเงิน) ลงสู่คอมพิวเตอร์
 - File -> Exit: ทำหน้าที่ปิดโปรแกรม (ระบบจะทำการบันทึกข้อมูลอัตโนมัติก่อนปิดเสมอ)
- ส่วนที่ 2: แผงควบคุมด้านซ้าย (Pet & Gacha Panel)
 - My Pets (รายการสัตว์เลี้ยง): แสดงรายชื่อสัตว์เลี้ยงทั้งหมดที่ผู้เล่นครอบครองพร้อมระดับเลเวล ผู้เล่นสามารถคลิกเลือกชื่อในลิสต์เพื่อดึงข้อมูลของสัตว์ตัวนั้นๆ มาดูแถบหน้าจอหลักได้
 - Open Gacha! (ปุ่มสุ่มกาชา): ทำหน้าที่เปิดหน้าต่างสุ่มรับสัตว์เลี้ยงตัวใหม่ โดยมีค่าใช้จ่ายจำนวน 100 เหรียญต่อการสุ่มหนึ่งครั้ง
- ส่วนที่ 3: แผงสถานะและการดูแลด้านขวา (Care & Status Panel)
 - ส่วนแสดงข้อมูลทั่วไป: แสดงรูปภาพ ชื่อ ประเภท (Type) เลเวล และข้อความบอกสถานะความรู้สึกของสัตว์เลี้ยง
 - หลอดพลังสถานะ (Progress Bars): ใช้ติดตามค่าพลังชีวิต (HP), ความหิว (Hunger), ความสุข (Happiness) และพลังงาน (Energy)
 - Feed (ปุ่มให้อาหาร): ใช้เงิน 20 เหรียญ เพื่อเพิ่มค่าความหิวและฟื้นฟูพลังชีวิต
 - Play (ปุ่มเล่นด้วย): ทำให้ผู้เล่นได้รับเงินสุ่ม (ประมาณ 30 เหรียญ) และค่าประสบการณ์ (EXP) แลกกับการลดลงของความหิวและพลังงาน
 - Sleep/Wake (ปุ่มหลับ/ตื่น): สั่งให้สัตว์เลี้ยงนอนหลับเพื่อฟื้นฟูพลังงาน โดยในสถานะนี้ค่าเลือดจะไม่ลดลงแม้จะหิว
 - Special (ปุ่มทักษะพิเศษ): สั่งให้สัตว์เลี้ยงแสดงท่าทางพิเศษหรือส่งเสียงร้องเฉพาะตัว
 - Rename (ปุ่มเปลี่ยนชื่อ): ทำหน้าที่เปิดหน้าต่างสำหรับตั้งชื่อใหม่ให้กับสัตว์เลี้ยง (รองรับเฉพาะภาษาอังกฤษ)

- ส่วนที่ 4: แถบสถานะด้านล่าง (Bottom Status Bar)
 - หน้าปัดจำนวนเหรียญ (Coins): แสดงยอดเงินปัจจุบันของผู้เล่น
 - หน้าต่างข้อความ (Status Message): แจ้งเตือนผลลัพธ์จากการกระทำ หรือแจ้งเตือนฉุกเฉินเมื่อสัตว์เสี่ยงต้องการการดูแล

3. ระบบและกลไกของเกม (Game Mechanics)

เพื่อสร้างความท้าทายและความสมจริง ตัวเกมมีกลไกการทำงานเบื้องหลังดังต่อไปนี้:

- 3.1 ระบบจำลองการใช้ชีวิต (Status Decay & Death): * ระบบทำงานผ่าน Thread "นาฬิกาชีวิต" ส่งผลให้ค่าความอึด ความสุข และพลังงานลดลงตามกาลเวลา (แม้ไม่ได้คลิกเลือกสัตว์ตัวนั้นอยู่ก็ตาม)
 - หากค่าสถานะตกสู่จุดวิกฤต (ความอึด < 20, ความสุข < 20, พลังงาน < 10) จะส่งผลให้หมดพลังชีวิต (HP) ลดลงตาม
 - หากค่า HP ลดลงเหลือ 0 สัตว์เลี้ยงจะเข้าสู่สถานะ Game Over และไม่สามารถตอบสนองต่อคำสั่งใดๆ ได้อีก
- 3.2 ระบบพัฒนาการ (Level System): * สัตว์เลี้ยงสะสมค่าประสบการณ์ (EXP) จากการทำกิจกรรม เริ่มต้นที่เลเวล 1 และสูงสุดที่เลเวล 50
 - สมการการเลื่อนระดับคือ EXP ที่ต้องการ = เลเวลปัจจุบัน x 100
 - เมื่อเลเวลอัป สัตว์เลี้ยงจะได้รับการฟื้นฟูค่าเลือด 10 หน่วยและเพิ่มความสุข 5 หน่วย
- 3.3 ระบบความน่าจะเป็น (Gacha Rates):
 - Cat (แมว) - อัตราการรอบ 50%: เป็นมิตร ดูแลง่าย หิวและเหนื่อยในอัตราปกติ
 - Dog (สุนัข) - อัตราการรอบ 40%: พลังงานสูง ได้รับค่าประสบการณ์จากการเล่นด้วยมากกว่าปกติ
 - MythicPet (สัตว์ตำนาน) - อัตราการรอบ 10%: มีความทนทานสูง มาพร้อม "Ultimate Skill" (ใช้ปุ่ม Special) ที่ช่วยฟื้นฟูเลือดจนเต็มได้ทันที (จำกัดการใช้ 3 ครั้งต่อตัว)
 - Pity System: มีระบบการันตีออกสัตว์ระดับ MythicPet เสมอ หากผู้เล่นทำการสุ่มครบ 10 ครั้ง

4. ภาพรวมการออกแบบโปรแกรมและ UML Diagram

โครงสร้างของระบบถูกออกแบบด้วยสถาปัตยกรรม Object-Oriented Programming (OOP) โดยแบ่งออกเป็น 3 แพคเกจหลัก ได้แก่:

- com.gachapet.data : ดูแลระบบบันทึกและการจัดการคลังสัตว์เลี้ยงของผู้เล่น
- com.gachapet.model : จัดการตรรกะและเงื่อนไขการใช้ชีวิตของสัตว์
- com.gachapet.t: ควบคุมหน้าจอแสดงผลผ่านเครื่องมือ JavaFX

รายละเอียดความสัมพันธ์ของคลาส สามารถพิจารณาได้จาก UML Diagram ด้านล่างนี้:

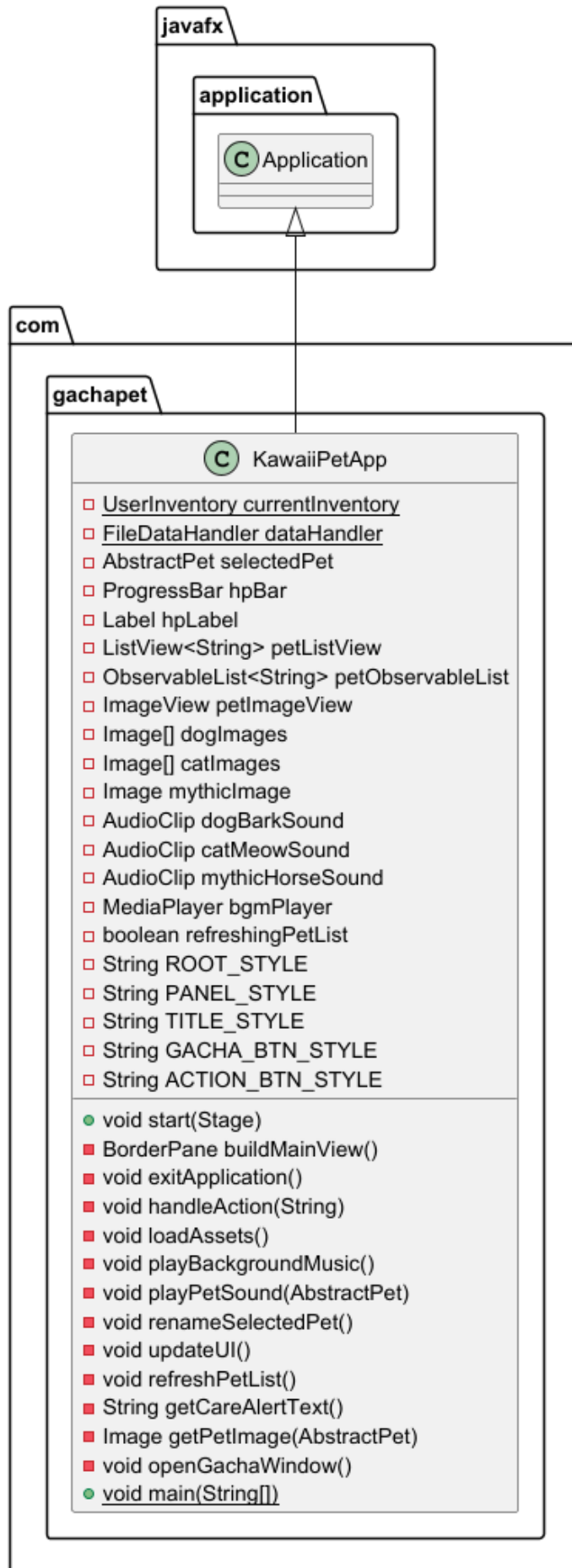
 UserInventory

- int STARTING_COINS
- int MAX_PET_CAPACITY
- ArrayList<AbstractPet> pets
- int coins
- String playerName
- «Create» UserInventory(String)
- boolean addPet(AbstractPet)
- AbstractPet removePet(int)
- AbstractPet getPet(int)
- List<AbstractPet> getAllPets()
- int getPetCount()
- boolean isFull()
- boolean isEmpty()
- boolean isValidIndex(int)
- int getCoins()
- void setCoins(int)
- void addCoins(int)
- boolean spendCoins(int)
- boolean hasEnoughCoins(int)
- String getPlayerName()
- void setPlayerName(String)
- ▲ ArrayList<AbstractPet> getPetsInternal()
- String toString()

 FileDataHandler

- String DEFAULT_SAVE_FILE
- String PREFIX_PLAYER
- String PREFIX_COINS
- String PREFIX_PET
- String filePath
- «Create» FileDataHandler()
- «Create» FileDataHandler(String)
- boolean saveData(UserInventory)
- UserInventory loadData()
- boolean deleteSaveFile()
- boolean hasSaveFile()
- AbstractPet parsePetFromCsv(String)
- AbstractPet createPetByType(String,String)
- void setInventoryCoins(UserInventory,int)
- String getFilePath()





gachapet

test

 PetLogicTest

☐ Cat cat
☐ Dog dog
☐ MythicPet mythic

▲ void setUp()
 ▲ void testHpCannotGoBelowZero()
 ▲ void testHpCannotExceedMax()
 ▲ void testHungerCannotGoBelowZero()
 ▲ void testHungerCannotExceedMax()
 ▲ void testHappinessCannotGoBelowZero()
 ▲ void testHappinessCannotExceedMax()
 ▲ void testEnergyCannotGoBelowZero()
 ▲ void testEnergyCannotExceedMax()
 ▲ void testIsAliveWhenHpIsZero()
 ▲ void testNewPetsAreAlive()
 ▲ void testInitialValues()
 ▲ void testConstructorThrowsOnInvalidName()
 ▲ void testPolymorphismMakeSound()
 ▲ void testGetPetType()
 ▲ void testGetEmoji()
 ▲ void testGetSpecialSkill()
 ▲ void testPolymorphismPlayDifference()
 ▲ void testMythicEatBetterThanDog()
 ▲ void testCatPlayChangesStats()
 ▲ void testDogPlayChangesStats()
 ▲ void testPerformActionPolymorphism()
 ▲ void testHungerDecayRateDifference()
 ▲ void testTickUpdatesAgeWithoutImmediateStatDecay()
 ▲ void testStatsDecayAfterCareInterval()
 ▲ void testInactivePetStatsDecaySlowly()
 ▲ void testCareAlertWhenStatsDropToHalf()
 ▲ void testLowStatsReduceHp()
 ▲ void testGoodStatsRestoreHp()
 ▲ void testSleepSetsSleepingTrue()
 ▲ void testWakeUpSetsSleepingFalse()
 ▲ void testSleepingPetGainsEnergy()
 ▲ void testSleepingPetCannotPerformNormalActions()
 ▲ void testMythicUltimateHealsToFull()
 ▲ void testMythicUltimateChargesDoNotExceedMax()

 GachaTest

☐ double TOLERANCE
☐ int SAMPLE_SIZE
☐ long TEST_SEED
☐ GachaSystem gacha

▲ void setUp()
 ▲ void testTotalDropRateEqualsOneHundredPercent()
 ▲ void testGachaCostIsPositive()
 ▲ void testMythicDropRateApproximatelyTenPercent()
 ▲ void testDogDropRateApproximatelyFortyPercent()
 ▲ void testCatDropRateApproximatelyFiftyPercent()
 ▲ void testRollNeverReturnsNull()
 ▲ void testRollReturnsValidPetType()
 ▲ void testRollReturnsAbstractPetSubtype()
 ▲ void testRolledPetHasValidInitialState()
 ▲ void testRollTenReturnsExactlyTenPets()
 ▲ void testRollTenNoNullElements()
 ▲ void testRollTenGuaranteesMythicPet()
 ▲ void testCanRoll()
 ▲ void testCanRollTen()
 ▲ void testGetDropRateInfoContainsAllPetTypes()

 InventoryTest

☐ String TEST_FILE
☐ UserInventory inventory
☐ FileDataHandler dataHandler

▲ void setUp()
 ▲ void tearDown()
 ▲ void testInitialCoins()
 ▲ void testInitialPetCount()
 ▲ void testPlayerName()
 ▲ void testAddPetSuccess()
 ▲ void testAddNullPetReturnsFalse()
 ▲ void testAddPetWhenInventoryFull()
 ▲ void testInventoryStoresDifferentPetTypes()
 ▲ void testGetAllPetsReturnsCopy()
 ▲ void testRemovePetSuccess()
 ▲ void testRemovePetInvalidIndex()
 ▲ void testGetPetInvalidIndex()
 ▲ void testSpendCoinsInsufficientFunds()
 ▲ void testSpendCoinsSuccess()
 ▲ void testAddCoins()
 ▲ void testInvalidCoinOperations()
 ▲ void testSetCoinsPreventsNegativeValues()
 ▲ void testHasEnoughCoins()
 ▲ void testSaveDataCreatesFile()
 ▲ void testLoadDataWithNoFile()
 ▲ void testHasSaveFile()
 ▲ void testDeleteSaveFile()
 ▲ void testSaveAndLoadRoundTrip()
 ▲ void testMythicPetSaveLoad()

5. การประยุกต์ใช้หลักการเชิงวัตถุ (OOP) ในโปรเจกต์

โปรเจกต์นี้มีการนำแนวคิด OOP มาประยุกต์ใช้อย่างครอบคลุมทั้ง 4 แกนหลัก โดยมีรายละเอียดการเรียกใช้งานดังนี้:

- 5.1 การสืบทอดคุณสมบัติ (Inheritance): * มีการสร้าง AbstractPet ให้ทำหน้าที่เป็นคลาสแม่ (Superclass)
 - มีการสร้างคลาสลูก (Subclasses) ได้แก่ Cat, Dog และ MythicPet ให้ทำการสืบทอด (extends) จาก AbstractPet
 - ทำให้คลาสลูกได้รับคุณสมบัติตัวแปร (เช่น hp, hunger, happiness, energy, level) และเมธอดการทำงาน (เช่น tick(), tickInactive()) จากคลาสแม่ไปโดยอัตโนมัติ ช่วยลดความซ้ำซ้อนของโค้ดลงได้อย่างมาก
- 5.2 การกำหนดมาตรฐานผ่านอินเตอร์เฟซ (Interface):
 - มีการสร้างคลาสอินเตอร์เฟซชื่อ Actionable เพื่อบังคับพฤติกรรมพื้นฐานของเกม
 - กำหนดให้ AbstractPet ทำการ implements คลาส Actionable
 - ส่งผลให้คลาสสัตว์เลี้ยงทุกประเภทจำเป็นต้องมีเมธอดหลักในการตอบสนองต่อผู้เล่นอย่างครบถ้วน ได้แก่ eat(), play(), sleep(), และ performAction()
- 5.3 การพ้องรูป (Polymorphism): มีการประยุกต์ใช้ 2 ส่วนหลัก ได้แก่
 - ในหน้าจอ UI: ทำการประกาศตัวแปร selectedPet ด้วยชนิดคลาสแม่ (AbstractPet) เมื่อผู้เล่นกดปุ่มให้อาหาร โปรแกรมจะสั่งงานผ่าน selectedPet.eat() ซึ่งตัวระบบ Java จะตรวจจับและเรียกใช้ฟังก์ชัน eat() ของแมว สุนัข หรือสัตว์ดำนาน ให้เองโดยอัตโนมัติตามประเภทที่ผู้เล่นเลือกอยู่ โดยไม่จำเป็นต้องใช้คำสั่ง if-else ตรวจสอบประเภท
 - ในระบบข้อมูล: ทำการใช้งาน ArrayList<AbstractPet> ทำให้สามารถบันทึกข้อมูลสัตว์เลี้ยงหลากหลายสายพันธุ์ปะปนกันภายในรายการ (List) เดียวกันได้
- 5.4 การห่อหุ้มและปกป้องข้อมูล (Encapsulation):
 - ทำการปกป้องตัวแปรสถานะที่สำคัญทั้งหมด (Data Hiding) โดยกำหนด Access Modifier เป็น private (เช่น private int hp)
 - ป้องกันไม่ให้คลาสอื่นๆ หรือระบบ UI ลักลอบแก้ไขค่าสถานะของสัตว์เลี้ยงได้โดยตรง
 - บังคับให้การเรียกดูหรือเปลี่ยนแปลงข้อมูล ต้องทำผ่านเมธอด Getter และ Setter (เช่น getHp(), setHp(), addExp()) เพื่อให้ระบบสามารถตรวจสอบเงื่อนไข (Boundary) ความถูกต้องได้ก่อนเสมอ

6. การทดสอบความถูกต้องของระบบด้วย JUnit

เพื่อให้แน่ใจว่าลอจิกต่างๆ ของตัวเกมสามารถทำงานได้อย่างสมบูรณ์ การทดสอบด้วย JUnit จึงถูกแจกแจงออกเป็น 3 คลาสหลัก ดังนี้:

- 6.1 คลาส GachaTest (ทดสอบระบบสุ่มภาษา):
 - ทดสอบอัตราการสุ่ม (Drop Rate): เพื่อตรวจสอบว่าเปอร์เซ็นต์การออกของสัตว์แต่ละประเภทถูกต้องตามที่ออกแบบไว้
 - ทดสอบชนิดของผลลัพธ์ (Return Types): เพื่อยืนยันว่าเมื่อสั่งรันระบบภาษา ผลลัพธ์ที่ได้ออกมาจะมีเพียงออบเจกต์ประเภท Cat, Dog, หรือ MythicPet เท่านั้น

- ทดสอบระบบการันตี (Pity System): เพื่อตรวจสอบว่าหากจำลองการสุ่มครบ 10 ครั้ง ระบบจะบังคับคืนค่าผลลัพธ์เป็น MythicPet อย่างถูกต้อง
- 6.2 คลาส InventoryTest (ทดสอบระบบคลังเก็บข้อมูล):
 - ทดสอบการตั้งค่าเริ่มต้น (Setup Game): เพื่อตรวจสอบความถูกต้องของระบบเมื่อสร้างผู้เล่นหรือเปิดเกมใหม่
 - ทดสอบการเพิ่มสัตว์เลี้ยง (Add Pet): เพื่อยืนยันว่าเมื่อได้รับสัตว์เลี้ยงใหม่ ระบบจะทำการอัปเดตข้อมูลลงในลิสต์ (Inventory) ได้อย่างสมบูรณ์
 - ทดสอบระบบบันทึกข้อมูล (Save/Load Progress): เพื่อตรวจสอบการทำงานของ FileDataHandler ผ่านการจำลองการเขียนและอ่านไฟล์ข้อมูล เพื่อการันตีว่าสถานะต่างๆ ไม่สูญหาย
- 6.3 คลาส PetLogicTest (ทดสอบตรรกะการใช้ชีวิต):
 - ทดสอบการป้องกันขอบเขตสถานะ (Boundary Protection): เพื่อตรวจสอบตรรกะการปิดเศษว่าค่าสถานะต่างๆ จะไม่ลดลงจนติดลบ และไม่เพิ่มขึ้นจนทะลุ 100
 - ทดสอบความเสื่อมถอยของสถานะ (Status Decay): เพื่อตรวจสอบระบบนาฬิกาชีวิตว่าค่าพลังงานและระดับความหิวลดลงถูกต้องเมื่อเวลาผ่านไป
 - ทดสอบการนอนหลับ (Sleep Behavior): เพื่อตรวจสอบว่าฟังก์ชันการสั่งให้สัตว์เลี้ยงเข้านอนสามารถรองรับการลดลงของเลือดและฟื้นฟูค่าพลังงานได้จริง
 - ทดสอบระบบประสบการณ์ (EXP & Level Up): เพื่อตรวจสอบสมการอัปเลเวลว่าเมื่อสัตว์เลี้ยงได้รับ EXP ตามเกณฑ์ จะสามารถเลื่อนระดับและได้รับโบนัสสถานะอย่างถูกต้อง
 - ทดสอบทักษะพิเศษ (Ultimate Skill): เพื่อทดสอบตรรกะเฉพาะตัวของ MythicPet ว่าระบบนับจำนวนครั้งการใช้สกิลถูกหักลบอย่างถูกต้อง และสามารถฟื้นฟูเลือดจนเต็ม 100 ได้จริง

7. วิธีการทดสอบรันโปรแกรม (Running the Application)

ผู้ประเมินผลสามารถทดสอบการรันโปรเจกต์ผ่านไฟล์ .jar ได้จาก 2 ช่องทาง ดังต่อไปนี้:

7.1 การรันผ่าน Command Line (Manual)

ทำการสร้างโฟลเดอร์ใหม่ นำไฟล์ .jar ไปวางไว้ด้านใน จากนั้นเปิด Command Prompt หรือ Terminal ขึ้นมาและป้อนคำสั่งดังต่อไปนี้:

```
java --module-path "path/to/javafx/lib" --add-modules javafx.controls,javafx.media -jar wewith-1-with-sources.jar
```

(หมายเหตุ: กรุณาเปลี่ยน path/to/javafx/lib ให้ตรงกับที่อยู่ไฟล์ไลบรารี JavaFX ในเครื่องของคุณ)

7.2 การรันผ่าน IntelliJ IDEA (Recommended)

1. ทำการสร้างโฟลเดอร์ใหม่ นำไฟล์ .jar ไปวางไว้ด้านใน แล้วเปิดโฟลเดอร์ดังกล่าวด้วยโปรแกรม IntelliJ IDEA
2. ไปที่เมนู File > Project Structure > Libraries และทำการเพิ่มโฟลเดอร์ไลบรารีของ JavaFX

3. ไปที่เมนู Run > Edit Configurations
4. ในช่อง VM Options ให้ระบุคำสั่งเชื่อมโยงโมดูลดังนี้:
`--module-path "path/to/javafx/lib" --add-modules javafx.controls,javafx.media`
5. ในช่อง JAR Application ให้ทำการเลือกไฟล์ .jar ของโปรเจกต์
6. คลิกปุ่ม Run เพื่อเริ่มต้นการทำงานของโปรแกรม

8. เอกสารคู่มืออ้างอิงโค้ด (Javadoc)

รายละเอียดเชิงลึกเกี่ยวกับพฤติกรรมและการทำงานของแต่ละคลาส เมธอด รวมถึงพารามิเตอร์ต่างๆ ภายในโปรเจกต์ ได้ถูกรวบรวมและจัดทำเป็นเอกสารอ้างอิงทางเทคนิค (Javadoc) อย่างเป็นระบบ สามารถเข้าถึงเอกสารดังกล่าวได้ผ่านทางช่องทางด้านล่างนี้:

ลิงก์สำหรับเข้าสู่หน้า Javadoc: <https://github.com/koalagirl-c/WeWithdrawProgmeth/tree/9ca503570c7de86b3097bba91313da0f1e7904a4/build/docs/javadoc>